

프리미엄 아카이브 — 데이터베이스 테이블 설계안

1. 한눈에 보기

- 테이블은 **딱 3개**입니다. 회원 계정은 Supabase Auth가 이미 관리하므로 우리가 만들 건 "글"과 "멤버십"뿐입니다.
- 가장 중요한 결정: **프리미엄 본문을 `posts` 에서 떼어내 `post_bodies` 라는 별도 테이블로 분리**합니다. 이유는 하나입니다 — PostgreSQL의 보안 규칙(RLS)은 "행" 단위로만 막을 수 있고 "컬럼" 단위로는 못 막기 때문입니다. 본문이 `posts` 안에 같이 있으면 DB 계층에서 본문만 가리는 게 불가능합니다.
- 맞보기는 **본문에서 잘라 쓰지 않고 `posts.preview` 라는 별도 컬럼**에 둡니다. 잘라 쓰려면 서버가 본문 전체를 일단 가져와야 하고, 그 순간 CLAUDE.md의 "보내놓고 가리지 않는다"가 깨집니다.
- 결제는 **별도 테이블을 만들지 않고 `memberships` 한 행에 결제 식별자 한 컬럼만** 남깁니다.
- 멤버십 상태는 `memberships` 테이블에 **행이 있으면 회원, 없으면 무료 회원**입니다. 불린 컬럼도, 만료일도 이번엔 없습니다.

테이블	왜 있는가 (한 줄)
<code>posts</code>	글의 공개 정보(제목·요약·대표 이미지·맞보기·무료/프리미엄)를 담는다. 홈 목록과 SEO가 이것만 읽는다.
<code>post_bodies</code>	글의 본문만 따로 담는다. 권한 검사를 걸 수 있는 유일한 방법이라서 분리했다.
<code>memberships</code>	누가 멤버십 회원인지 기록한다. 행이 있으면 회원이다.

2. 전제

이 설계는 다음을 전제합니다. 전제가 바뀌면 설계도 바뀝니다.

- 데이터베이스는 **Supabase(PostgreSQL)**를 쓴다.
- 회원 계정은 **Supabase Auth가 관리**한다. 이메일·비밀번호·비밀번호 해시·가입일은 전부 Supabase가 만들어 주는 `auth.users` 테이블에 이미 들어 있습니다. 그래서 우리는 `users` 나 `profiles` 같은 테이블을 **만들지 않습니다**. 내 서재에 보여줄 이메일도 로그인 세션에서 바로 꺼내 씁니다.
- RLS(Row Level Security, 행 수준 보안)**를 켜다. 풀어 쓰면 "이 표의 각 줄을, 지금 요청한 사람이 볼 수 있는지 데이터베이스가 직접 판단하는 기능"입니다. 화면 코드가 아니라 데이터베이스가 판단하므로, 프론트엔드를 우회해 API를 직접 호출해도 막힙니다.

4. 브라우저에서 쓰는 키는 공개 키(anon key)다. 이 키는 누구나 볼 수 있다고 가정합니다. 그래서 보안은 전적으로 RLS에 걸립니다.
 5. `service_role` 키는 서버에서만 쓴다. 이 키는 RLS를 통째로 무시하므로, 결제 확인 후 멤버십을 기록하는 서버 코드에서만 씁니다. 절대 브라우저로 내려보내지 않습니다.
 6. 운영자는 글을 DB에 직접 넣는다. (PRD: 관리 화면은 범위 밖) 따라서 글 작성/수정을 위한 테이블·컬럼·권한은 만들지 않습니다.
 7. 결제는 테스트 모드 일회성이다. (CLAUDE.md) 정기 결제 갱신·해지 처리는 범위 밖입니다.
-

3. 테이블별 설계

3-1. `posts` — 글의 공개 정보

누구나 읽어도 되는 정보만 들어갑니다. 여기에는 본문이 없습니다.

컬럼명	타입	필수	설명 / PRD 어느 화면·기능 때문에 필요한가
id	uuid	필수 (PK)	글 하나를 가리키는 고유 번호. <code>post_bodies</code> 가 이걸로 본문을 연결한다.
slug	text	필수 (유니크)	주소에 들어가는 영문 이름(예: <code>/posts/first-archive-note</code>). 운영에 필요한 것 > 검색 노출(SEO) — 검색 엔진은 의미 있는 주소를 선호한다.
title	text	필수	글 제목. 화면 1 홈 (목록에 제목 표시), 화면 2 글 상세, SEO/공유 미리보기 .
summary	text	필수	한두 줄 요약. 화면 1 홈 (목록에 요약 표시), SEO/공유 미리보기 설명 .
cover_image_url	text	선택	대표 이미지 주소. 운영에 필요한 것 > SEO ("대표 이미지가 공유 시 제대로 보인다"). 이미지 파일 자체는 Supabase Storage나 외부 주소에 두고 주소만 저장한다.
is_premium	boolean	필수 (기본 <code>false</code>)	프리미엄 글인지. 화면 1 홈 (무료/프리미엄 배지, 자물쇠 표시), 화면 2 글 상세 (전문 vs 맛보기 분기), 그리고 RLS 정책의 판단 기준 .
preview	text	필수	앞부분 맛보기. 화면 2 글 상세 ("프리미엄 글 + 멤버십 없음: 앞부분 맛보기까지만"). 무료 글에도 값을 넣어 두면 목록·공유 미리보기에 그대로 쓸 수 있다.
published_at	timestampz	필수 (기본 현재시각)	공개 시각. 화면 1 홈 ("글 목록을 최신순으로") 정렬 기준.

넣지 않은 것: `author_id` (운영자 1명, 관리 화면 없음), `view_count` (PRD에 조회수 화면 없음 — 방문자 분석은 GA4가 한다), `category` / `tags` (범위 밖), `status` / `is_draft` (운영자가 직접 넣으므로 넣는 순간이 곧 공개), `updated_at` (수정 이력을 보여주는 화면이 없다).

3-2. `post_bodies` — 본문

본문만 따로 사는 테이블입니다. 이 테이블 하나가 이 설계의 보안 장치 전부입니다.

컬럼명	타입	필수	설명 / PRD 어느 화면·기능 때문에 필요한가
post_id	uuid	필수 (PK, posts.id 참조)	어느 글의 본문인지. 글 하나에 본문 하나(1:1).
body	text	필수	글 전문. 화면 2 글 상세 ("무료 글: 전문", "프리미엄 글 + 멤버십 있음: 전문"). 권한 없는 사람에게는 이 행 자체가 조회되지 않는다.

3-3. memberships — 멤버십 회원 명단

컬럼명	타입	필수	설명 / PRD 어느 화면·기능 때문에 필요한가
user_id	uuid	필수 (PK, auth.users.id 참조)	누구의 멤버십인지. PK로 두면 한 사람이 두 줄 생기는 사고가 DB 차원에서 막힌다. 화면 4 멤버십 안내 ("이미 멤버십 회원이면 '이미 이용 중입니다'").
started_at	timestampz	필수 (기본 현재시각)	언제부터 회원인지. 화면 5 내 서재 에 "언제부터 이용 중" 한 줄을 붙일 수 있고, 결제 문의가 왔을 때 시점 대조에 쓴다.
payment_ref	text	필수	결제 대행사가 준 결제 식별자(테스트 모드 값). 화면 4 멤버십 안내 의 결제 결과를 되짚을 수 있는 유일한 끈. 아래 4-5 참고.

여기에 이메일을 복사해 두지 않습니다. 이메일은 auth.users 에 이미 있고, 두 군데 두면 언젠가 반드시 서로 달라집니다.

4. 설계 판단이 갈린 지점과 선택 근거

4-1. 무료/프리미엄 구분을 어떻게 표현할까

선택지	장점	단점	판단
A. <code>is_premium boolean</code>	한 칸. 조건문이 단순. RLS 정책도 한 줄.	나중에 등급이 3개 이상이 되면 못 늘린다.	추천
B. <code>tier text</code> ('free'/'premium')	등급 추가가 쉽다.	오타가 값으로 들어간다('Premium' , 'preimum'). 막으려면 CHECK 제약이 또 필요. 지금 등급은 2개다.	보류
C. 별도 <code>plans</code> 테이블 + 참조	가장 유연.	행 2개짜리 테이블과 JOIN을 평생 달고 다닌다. PRD에 요금제는 하나뿐이다.	과설계

→ A. `is_premium boolean`. PRD의 배지는 "무료/프리미엄" 두 가지뿐이고, 화면 2의 분기도 둘뿐입니다. 등급이 실제로 셋이 되는 날 `tier` 로 바꾸는 마이그레이션은 10분짜리입니다.

4-2. 맛보기 — 별도 컬럼인가, 본문에서 잘라 쓰는가

선택지	어떻게 동작하나	문제
A. 본문에서 앞 N글자를 잘라 쓴다	서버가 <code>body</code> 를 통째로 읽어 앞부분만 화면에 보낸다.	치명적. 본문을 읽어야 자를 수 있으니, DB는 비회원에게도 본문 조회를 허용해야 한다. RLS로 막을 수가 없다. CLAUDE.md의 "권한 없는 사람에게 아예 보내지 않는다"와 정면 충돌. 또 문단 중간이 잘려 문장이 깨지고, 마크다운이면 태그가 열린 채 끊긴다.
B. <code>posts.preview</code> 별도 컬럼	운영자가 맛보기 문단을 직접 골라 넣는다. 공개 테이블에 있으므로 누구나 읽는다.	운영자가 글 하나당 두 곳을 채워야 한다. 앞부분을 수정하면 맛보기도 손봐야 한다.
C. 본문을 <code>preview</code> + <code>rest</code> 두 컬럼으로 쪼갠다	이어 붙여 전문을 만든다.	컬럼 두 개가 같은 테이블에 있으면 RLS로 <code>rest</code> 만 못 가린다(4-3 참고). 결국 테이블을 또 나눠야 해서 B와 같아진다.

→ B. 별도 `preview` 컬럼. 유일하게 "본문을 아예 안 보내는" 구현이 가능한 선택지입니다. 운영자가 두 군데를 채우는 수고는, 대신 맛보기를 어디서 끊을지 사람이 고르게 되는 이득으로 상쇄됩니다 — 자동으로

자른 맛보기보다 사람이 고른 맛보기가 훨씬 잘 팔립니다.

4-3. 본문을 `posts` 안에 둘까, 별도 테이블로 뺄까 (가장 중요한 결정)

선택지	판단
A. <code>posts.body</code> 컬럼 + RLS	불가능. RLS는 이름 그대로 Row(행) 단위입니다. "이 행은 보여주되 <code>body</code> 칸만 가려라"는 RLS로 표현할 수 없습니다. 프리미엄 글 행을 통째로 숨기면 홈 목록에서 그 글이 사라져 버립니다(PRD 화면 1은 프리미엄 글도 목록에 보여야 합니다).
B. <code>posts.body</code> + 컬럼 권한 (<code>GRANT SELECT (제한된 컬럼 목록)</code>)	이론상 가능하지만, 컬럼을 추가할 때마다 GRANT를 다시 손봐야 하고, <code>select *</code> 한 번이면 전체가 권한 오류로 죽습니다. 실수 한 번의 대가가 "본문 유출"이라 권한이 없습니다.
C. <code>posts.body</code> + 공개용 뷰 (view)	동작은 하지만 뷰·기본 테이블 양쪽 권한을 다 신경 써야 하고, 초보자가 읽었을 때 어디가 잠겨 있는지 한눈에 안 보입니다.
D. 본문을 <code>post_bodies</code> 별도 테이블로 분리	본문이 독립된 "행"이 되므로 RLS가 원래 잘하는 일(행 숨기기)로 딱 떨어집니다. "본문 테이블은 조건을 만족할 때만 읽힌다" — 규칙이 한 문장입니다.

→ D. RLS가 행 단위라는 제약이 설계를 결정합니다. 테이블을 하나 더 만드는 비용보다, "본문은 저 테이블에만 있고 저 테이블엔 자물쇠가 걸려 있다"고 한 줄로 말할 수 있는 가치가 큼니다.

4-4. 멤버십 상태를 어떻게 표현할까

선택지	장점	단점	판단
A. <code>profiles.is_member boolean</code>	제일 단순해 보인다.	이것 하나 때문에 <code>profiles</code> 테이블을 새로 만들어야 한다. 게다가 가입만 하고 결제 안 한 사람도 행이 생겨야 해서, 가입 시 행을 자동 생성하는 트리거가 따라붙는다. 언제 회원이 됐는지 모른다.	탈락
B. <code>profiles.membership_expires_at</code> (만료일)	기간제 구독의 정석.	PRD에 갱신도 해지도 없다 (범위 밖). 테스트 모드 일회 결제라 넣을 만료일이 없어서, 결국 <code>2099-12-31</code> 같은 가짜 값을 넣게 된다. 가짜 값은 언젠가 진짜처럼 읽힌다.	보류
C. 별도 <code>memberships</code> 테이블, 행이 있으면 회원	<code>profiles</code> 테이블이 통째로 불필요해진다. 트리거도 불필요(결제 성공 시에만 행 삽입). 언제 결제했는지가 자연히 남는다. RLS 정책이 "내 행이 있나?" 한 줄이 된다. 기간제로 갈 때는 컬럼 하나 (<code>expires_at</code>) 추가로 끝.	테이블이 하나 는다.	추천

→ C. "회원이다"를 **행의 존재**로 표현합니다. 켜고 끄는 불린 값보다 "결제라는 사건이 있었다"는 기록이 사실에 가깝고, 한 사람이 두 번 결제해도 PK가 막아 줍니다.

기간제로 확장할 때: `alter table memberships add column expires_at timestamptz;` 를 더하고 RLS 정책의 조건에 `and (m.expires_at is null or m.expires_at > now())` 를 붙이면 됩니다. **지금은 넣지 않습니다** — 쓰지 않는 컬럼은 버그입니다.

4-5. 결제 기록을 남길까

선택지	판단
A. <code>payments</code> 테이블(금액·통화·상태·결제수단·영수증·시도 이력)	만들지 않습니다. PRD에 결제 내역 화면이 없고(내 서재는 "멤버십 상태"만 보여줍니다), 관리자 화면·환불·해지가 전부 범위 밖입니다. 게다가 이 데이터의 진짜 원본은 결제 대행사입니다. 우리가 복사해 두면 대행사 쪽에서 환불이 일어났을 때 우리 표만 거짓말을 하게 됩니다.
B. 아무것도 안 남긴다	위험합니다. 결제는 됐는데 우리 DB 쓰기가 실패하는 사고가 반드시 한번은 납니다. 그때 "이 사람이 정말 결제했나"를 대조할 끈이 전혀 없으면 수동 복구가 불가능합니다.
C. <code>memberships.payment_ref</code> 컬럼 하나	결제 대행사 대시보드에서 그 값으로 검색하면 금액·시각·상태를 전부 볼 수 있습니다. 돈 이야기는 원본에게 묻고, 우리는 원본을 가리키는 끈 하나만 보관합니다.

→ C. 별도 테이블은 없음, 컬럼 하나. CLAUDE.md는 보안·데이터 손실 방지를 간소화 대상에서 제외하므로, "아무것도 안 남기기"는 고르지 않았습니다.

5. 권한 설계 — 프리미엄 본문 보호 (RLS)

5-1. 용어 한 줄 풀이

- **RLS (Row Level Security, 행 수준 보안):** 표의 각 줄을 지금 요청한 사람이 볼 수 있는지 데이터베이스가 직접 판단하는 기능. 화면 코드가 아니라 DB가 판단합니다.
- **정책(policy):** "어떤 줄을 누가 볼 수 있다"를 적어 둔 규칙 한 개.
- `auth.uid()` : 지금 요청을 보낸 사람의 회원 번호를 돌려주는 Supabase 함수. 로그인하지 않았으면 비어 있습니다(`null`).
- `anon` / `authenticated` : 각각 "로그인 안 한 방문자" / "로그인한 회원"을 가리키는 역할 이름.
- `service_role` : 서버 전용 열쇠. **RLS를 전부 무시합니다.** 브라우저에 절대 내려보내면 안 됩니다.

5-2. 방어의 구조 (세 줄 요약)

1. `posts` 에는 본문이 없다 → 목록과 맛보기는 마음껏 공개해도 안전하다.
2. `post_bodies` 는 무료 글이거나, 요청자가 멤버십 회원일 때만 행이 조회된다 → 권한 없는 사람에게는 본문이 전송 자체가 안 된다.
3. `memberships` 에는 누구도 직접 행을 넣을 수 없다 → 사용자가 자기 자신을 회원으로 만들 수 없다.

3번이 특히 중요합니다. 본문 정책이 아무리 튼튼해도 사용자가 `memberships` 에 자기 행을 INSERT할 수 있으면, 결제 없이 회원이 되어 본문 정책을 정당하게 통과해 버립니다. `memberships` 에 **INSERT/UPDATE/DELETE 정책을 하나도 만들지 않는 것**이 방어입니다. RLS가 켜진 테이블은 허용 정책이 없으면 전부 거부이므로, 오직 `service_role` (결제 확인을 마친 서버 코드)만 행을 넣을 수 있습니다.

5-3. 정책 목록

테이블	동작	대상	조건	왜
<code>posts</code>	SELECT	모두(<code>anon</code> , <code>authenticated</code>)	무조건 허용	홈 목록·배지·맞보기·SEO는 전부 공개 정보. 본문이 없으니 공개해도 잃을 게 없다.
<code>posts</code>	INSERT/UPDATE/DELETE	—	정책 없음 = 전면 금지	운영자는 <code>service_role</code> 로 넣는다. 관리 화면은 범위 밖.
<code>post_bodies</code>	SELECT	모두	해당 글이 무료이거나 또는 요청자에게 멤버십 행이 있음	화면 2의 세 가지 분기를 DB가 그대로 집행한다.
<code>post_bodies</code>	INSERT/UPDATE/DELETE	—	정책 없음 = 전면 금지	같은 이유.
<code>memberships</code>	SELECT	<code>authenticated</code>	자기 행만 (<code>user_id = auth.uid()</code>)	화면 5 내 서재의 멤버십 상태 표시, 화면 4의 "이미 이용 중입니다" 판단. 남의 결제 여부는 볼 수 없다.
<code>memberships</code>	INSERT/UPDATE/DELETE	—	정책 없음 = 전면 금지	결제하지 않고 회원이 되는 경로를 원천 차단. 서버가 결제 성공을 확인한 뒤 <code>service_role</code> 로만 넣는다.

5-4. 주의할 함정 세 가지

1. `service_role` 키가 브라우저로 새면 위의 모든 정책이 무의미합니다. 이 키는 서버 환경 변수에만 두고, `NEXT_PUBLIC_` 같은 접두사를 절대 붙이지 마십시오. (CLAUDE.md 보안 규칙)

2. 화면에서 `if (isMember)` 로 본문을 가리는 코드는 방어가 아닙니다. 그건 그냥 보기 좋게 만드는 것이고, 실제 방어는 위의 `post_bodies` 정책 한 줄입니다. 화면 코드가 통째로 틀려도 본문은 새지 않아야 하고, 이 설계에서는 그렇습니다.
3. 서버에서 `service_role` 로 글 상세를 읽어 오면 RLS가 꺼집니다. 글 상세 페이지는 반드시 사용자 세션 (공개 키)으로 조회해야 합니다. 서버 렌더링을 하더라도 사용자 토큰을 실어 보내십시오. 그러면 비회원이 프리미엄 글을 열 때 `post_bodies` 조회 결과가 0건으로 돌아오고, 그 0건이 바로 "맛보기만 보여줘라"는 신호가 됩니다.

5-5. 확인 방법 (배포 전 1분)

로그아웃 상태로 브라우저 콘솔에서 본문 테이블을 직접 때려 봅니다.

```
// 로그아웃 상태
await supabase.from('post_bodies').select('*')
// 기대 결과: 무료 글의 본문만 돌아온다. 프리미엄 글은 한 줄도 없어야 한다.

await supabase.from('memberships').insert({ user_id: '아무-uuid', payment_ref: 'x' })
// 기대 결과: 에러(정책 위반). 성공하면 설계가 뚫린 것이다.
```

6. 바로 쓰는 SQL

Supabase 대시보드 → SQL Editor에 그대로 붙여 넣고 실행하면 됩니다.

```

-- =====
-- 프리미엄 아카이브 : 테이블 + 보안 정책
-- =====

-- ----- 1. 글의 공개 정보 (본문 없음) -----
create table public.posts (
    id            uuid            primary key default gen_random_uuid(),
    slug          text           not null unique,
    title         text           not null,
    summary       text           not null,
    cover_image_url text,
    is_premium    boolean        not null default false,
    preview       text           not null,
    published_at  timestamptz    not null default now()
);

-- 홈 목록은 항상 최신순 정렬이므로 인덱스를 하나 둔다.
create index posts_published_at_idx on public.posts (published_at desc);

-- ----- 2. 본문 (권한이 걸리는 유일한 테이블) -----
create table public.post_bodies (
    post_id uuid primary key references public.posts(id) on delete cascade,
    body    text not null
);

-- ----- 3. 멤버십 회원 명단 -----
create table public.memberships (
    user_id    uuid            primary key references auth.users(id) on delete cascade,
    started_at timestamptz    not null default now(),
    payment_ref text           not null
);

-- =====
-- RLS 켜기 : 켜는 순간 "정책에 적힌 것 외에는 전부 금지"가 된다.
-- =====
alter table public.posts      enable row level security;
alter table public.post_bodies enable row level security;
alter table public.memberships enable row level security;

-- ----- posts : 누구나 읽는다 (본문이 없으므로 안전) -----
create policy "글 목록과 맛보기는 누구나 읽는다"
    on public.posts
    for select
    to anon, authenticated
    using (true);

-- ----- post_bodies : 무료 글이거나, 멤버십 회원일 때만 -----
create policy "본문은 무료 글이거나 멤버십 회원에게만 보인다"

```

```

on public.post_bodies
for select
to anon, authenticated
using (
  -- (가) 이 본문이 달린 글이 무료 글인 경우
  exists (
    select 1
    from public.posts p
    where p.id = post_bodies.post_id
    and p.is_premium = false
  )
or
  -- (나) 요청한 사람이 멤버십 회원인 경우
  exists (
    select 1
    from public.memberships m
    where m.user_id = auth.uid()
  )
);

```

-- ----- memberships : 자기 것만 읽는다. 쓰기 정책은 일부러 없음 -----

create policy "내 멤버십 상태만 내가 읽는다"

```

on public.memberships
for select
to authenticated
using (user_id = auth.uid());

```

-- 주의: memberships 에 insert/update/delete 정책을 만들지 않는 것이 핵심 방어다.

-- 결제 성공을 확인한 서버 코드가 service_role 키로만 행을 넣는다.

-- 여기에 "본인이 본인 행을 넣을 수 있다" 정책을 추가하는 순간,

-- 누구나 결제 없이 프리미엄 회원이 된다.

초기 샘플 데이터

```
-- ----- 무료 글 1개 -----  
with new_post as (  
  insert into public.posts (slug, title, summary, cover_image_url, is_premium, preview)  
  values (  
    'why-i-started-this-archive',  
    '내가 이 아카이브를 시작한 이유',  
    '3년간 모아 온 자료를 왜 공개하기로 했는지에 대한 이야기.',  
    'https://example.com/covers/why-i-started.jpg',  
    false,  
    '처음에는 그냥 내 메모였습니다. 폴더 하나에 아무렇게나 쌓여 가던 것이...'  
  )  
  returning id  
)  
insert into public.post_bodies (post_id, body)  
select id,  
'처음에는 그냥 내 메모였습니다. 폴더 하나에 아무렇게나 쌓여 가던 것이...'  
  
(무료 글이므로 이 본문 전체가 누구에게나 보입니다. 비회원도, 검색 엔진도 읽습니다.)'  
from new_post;  
  
-- ----- 프리미엄 글 1개 -----  
with new_post as (  
  insert into public.posts (slug, title, summary, cover_image_url, is_premium, preview)  
  values (  
    'the-archive-method',  
    '아카이브 정리법 전문',  
    '자료를 버리지 않으면서도 찾을 수 있게 만드는 다섯 단계.',  
    'https://example.com/covers/archive-method.jpg',  
    true,  
    '자료를 모으는 일은 쉽습니다. 어려운 건 3년 뒤에 그걸 다시 찾는 일이죠. 제가 쓰는 방법은 다섯 단계입  
니다. 첫 번째는...'  
  )  
  returning id  
)  
insert into public.post_bodies (post_id, body)  
select id,  
'자료를 모으는 일은 쉽습니다. 어려운 건 3년 뒤에 그걸 다시 찾는 일이죠. 제가 쓰는 방법은 다섯 단계입니  
다. 첫 번째는...'  
  
(여기부터가 멤버십 회원 전용입니다. 비회원이 이 글을 열면 이 행은 조회 결과에  
아예 포함되지 않습니다. 화면에서 가리는 게 아니라, 데이터가 전송되지 않습니다.)'  
from new_post;
```

멤버십 행은 샘플로 넣지 않습니다. 넣으려면 실제 가입된 회원의 `user_id`가 필요하고, 그건 화면 3(회원가입)을 거쳐 만들어집니다. 테스트할 때는 가입 후 Supabase 대시보드에서 `auth.users`의 id

를 복사해 `insert into public.memberships (user_id, payment_ref) values ('복사한-uuid', 'test_manual');` 로 한 줄 넣어 보면, 같은 프리미엄 글이 전문으로 열리는 것을 확인할 수 있습니다. (PRD 완성 기준의 마지막 항목 점검)

7. 이번 범위에서 일부러 뺀 것

뺀 것	이유
<code>users</code> / <code>profiles</code> 테이블	Supabase Auth의 <code>auth.users</code> 가 이메일·비밀번호를 이미 관리합니다. 내 서재의 이메일도 로그인 세션에서 꺼냅니다. 복사본을 만들면 두 값이 언젠가 어긋납니다.
<code>payments</code> 테이블	결제 내역 화면이 PRD에 없습니다. 원본은 결제 대행사에 있고, 우리는 <code>payment_ref</code> 한 컬럼으로 그걸 가리킵니다.
<code>membership_expires_at</code> , <code>status</code> , <code>canceled_at</code>	구독 해지 화면이 범위 밖이고, 테스트 모드 일회 결제라 채울 진짜 값이 없습니다. 기간제로 갈 때 컬럼 하나 추가하면 됩니다.
<code>comments</code> , <code>likes</code> 테이블	PRD "이번 단계에서 하지 않는 것".
<code>categories</code> , <code>tags</code> , <code>post_tags</code>	같은. 목록은 최신순 하나뿐입니다.
전문 검색 인덱스(<code>tsvector</code> 등)	검색창이 범위 밖입니다.
<code>author_id</code> , <code>authors</code> 테이블	운영자 한 명이 DB에 직접 넣습니다. 관리 화면도 범위 밖입니다.
<code>view_count</code> , <code>read_at</code> , <code>reading_history</code>	방문자 분석은 GA4가 합니다. PRD에 조회수를 보여주는 화면이 없습니다.
<code>is_draft</code> / <code>status</code> (초안 관리)	글쓰기 에디터가 범위 밖이라 초안이라는 상태 자체가 생기지 않습니다. 운영자가 INSERT하는 순간이 곧 공개입니다.
<code>notifications</code> 테이블	알림이 범위 밖입니다. 가입 환영 메일은 Supabase Auth가 보내므로 테이블이 필요 없습니다.
<code>updated_at</code> 컬럼 전반	수정 시각을 보여주거나 쓰는 화면·기능이 하나도 없습니다. 필요해지는 날 추가합니다.

요약 3줄 — 테이블 3개(`posts` , `post_bodies` , `memberships`)로 PRD 다섯 화면이 전부 돌아갑니다. 보안의 전부는 본문을 별도 테이블로 떼어내 RLS를 건 것과, `memberships` 에 쓰기 정책을 하나도 만들지 않은 것 두

가지입니다. 만료일·결제 테이블·프로필 테이블은 지금 쓰지 않으므로 넣지 않았고, 필요해지는 날 각각 컬럼 하나 또는 테이블 하나로 추가됩니다.